

L08: 3D Rendering

Chris Carvelli

Original Lecture plan

⇒ 03/11 L09	Engine and Game architecture	01/12	Supervision
10/11 L10	Performance	08/12	Supervision
17/10 L11	Advanced topic 1 (TBD)		
24/11 L12	Advanced topic 2 (TBD)		

Updated Lecture plan

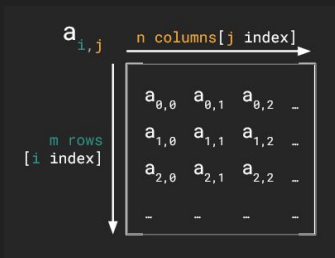
⇒ 03/11 L08	3D Rendering	01/12	Supervision OR Advanced topic
10/11 L09	Engine and Game architecture	08/12	Supervision
17/11 L10	Performance	15/11	Project Supervision (extra)
24/10 L11	Advanced topic		

Updated Lecture plan

⇒ 03/11 L08	3D Rendering	<u>01/12</u>	<u>Supervision OR Advanced topic</u>
10/11 L09	Engine and Game architecture	08/12	Supervision
17/11 L10	Performance	15/11	Project Supervision (extra)
<u>24/10 L11</u>	<u>Advanced topic</u>		

POLL THIS WEEK!

Recap



Matrices

```
struct Transform3D
{
    // navigational refs for efficient traversal
    // moving a transform in the hierarchy becomes
    // slightly more costly, but that's an operation
    // that should happen relatively rarely
    Transform3D* parent;
    Transform3D* child_first;
    Transform3D* child_last;
    Transform3D* sibling_prev;
    Transform3D* sibling_next;

    glm::mat4 local;
    glm::mat4 global;
};
```

3D Transform

```
#include <glm/glm.hpp>

int main() {
    glm::vec2 v(5,4);
    glm::mat2 m(1);
    glm::vec2 res = m * v;
}
```

glm

Binary dump

```
Transform transforms[1024];
int transforms_count;

PlayerData players[1024];
int players_count;

// write
SOI_WriteString("p_x" = SOI_ReadFile("scene_data", "x"));
SOI_Write2(fp_w, transforms, sizeof(transform) * transforms_count);
SOI_Write2(fp_w, players, sizeof(PlayerData) * players_count);

// read
SOI_ReadString("p_x" = SOI_ReadFile("scene_data", "x"));
SOI_Read2(fp_r, players, sizeof(PlayerData) * players_count);
SOI_Read2(fp_r, transforms, sizeof(transform) * transforms_count);
```

Key-based values (example: json)

```
{
  "components": [
    {
      "type": "R",
      "name": "transform",
      "data": [ ... ]
    },
    {
      "type": "K",
      "name": "PlayerData",
      "element_size": 48,
      "data": [
        {
          "speed_ground": 5.5,
          "speed_air": 4.5,
          "gravity": 7.5,
          ...
        }
      ]
    }
  ]
}
```

serialization

The background image is a 3D rendered scene from the video game 'The Legend of Zelda: Breath of the Wild'. It depicts a stone temple with a large, stylized fox statue on a high ledge. A small fox character, likely Link in his fox form, is standing on a set of stairs leading up to the temple. The scene is characterized by its low-poly, geometric style and vibrant colors, including a bright yellow door and a blue sky. A semi-transparent dark blue banner is overlaid across the center of the image, containing the text 'W6 - 3D rendering' in white.

W6 - 3D rendering

Agenda

1. Additional math
2. 3D Rendering
3. 3D Animation

Agenda

1. Additional math
2. 3D Rendering
3. 3D Animation

Normalize a vector

- Often we are interested in working with vectors of length of 1.0 (aka. unit vectors)
- Commonly used in many formulas
- We obtain unit vectors by dividing vectors with its own length

```
// normalize.cpp
#include <glm/glm.hpp>
using namespace glm;

vec3 x(10,0,0);
vec3 xn = normalize(x);           // (1, 0, 0)
vec3 xn_verbose = x / length(x); // (1, 0, 0)
```



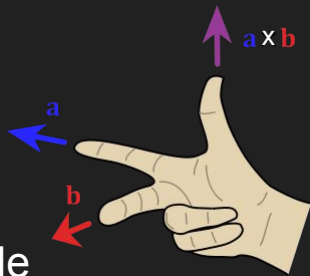
v (length = $\|v\|$)



$v_{\text{norm}} = v / \|v\|$

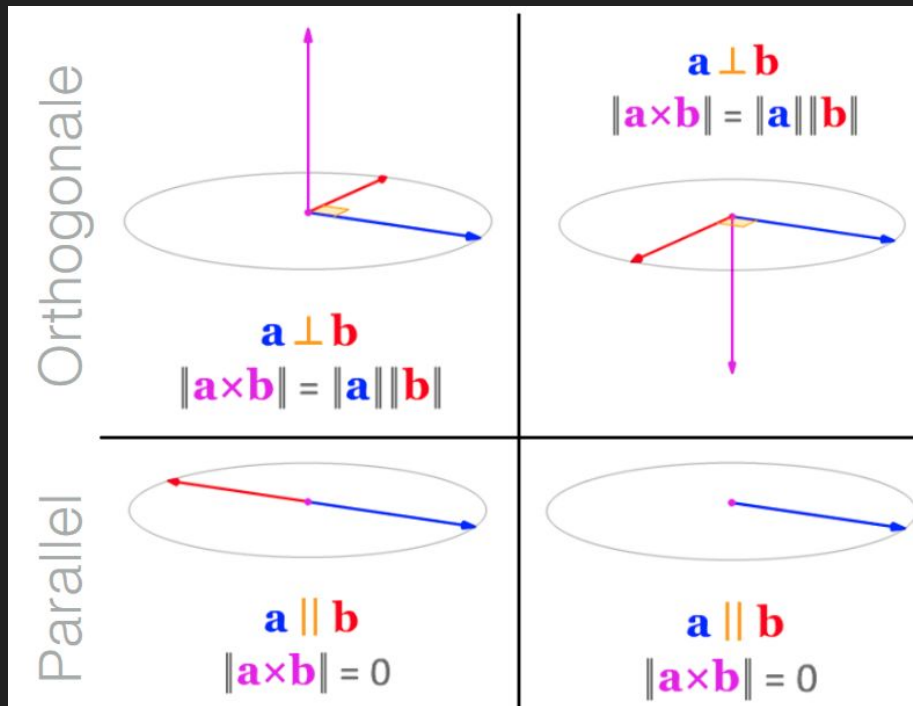
Cross product

- Only defined in 3D
- Takes two vectors and returns a vector
- Orientation defined with the right-hand rule



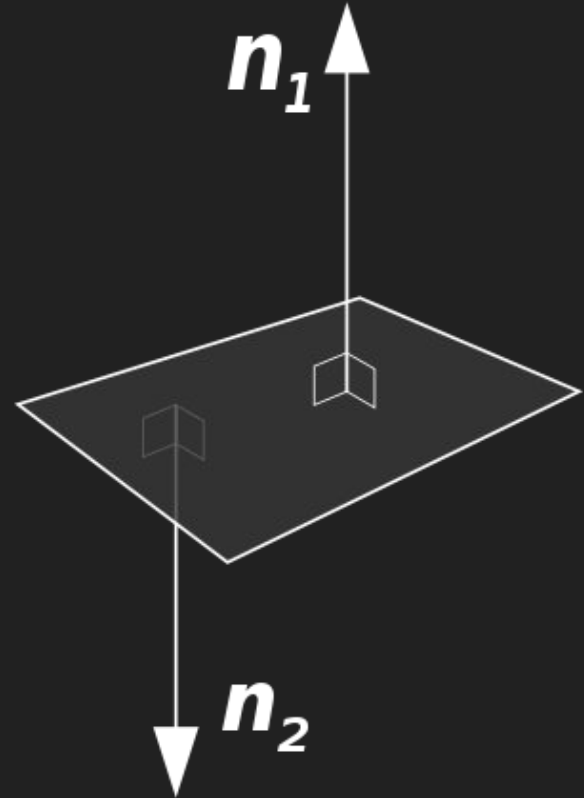
```
// cross_product.cpp
#include <glm/glm.hpp>
using namespace glm;

vec3 a(1,0,0);
vec3 b(0,0,-1);
vec3 c = cross(a,b); // (0,1,0)
```



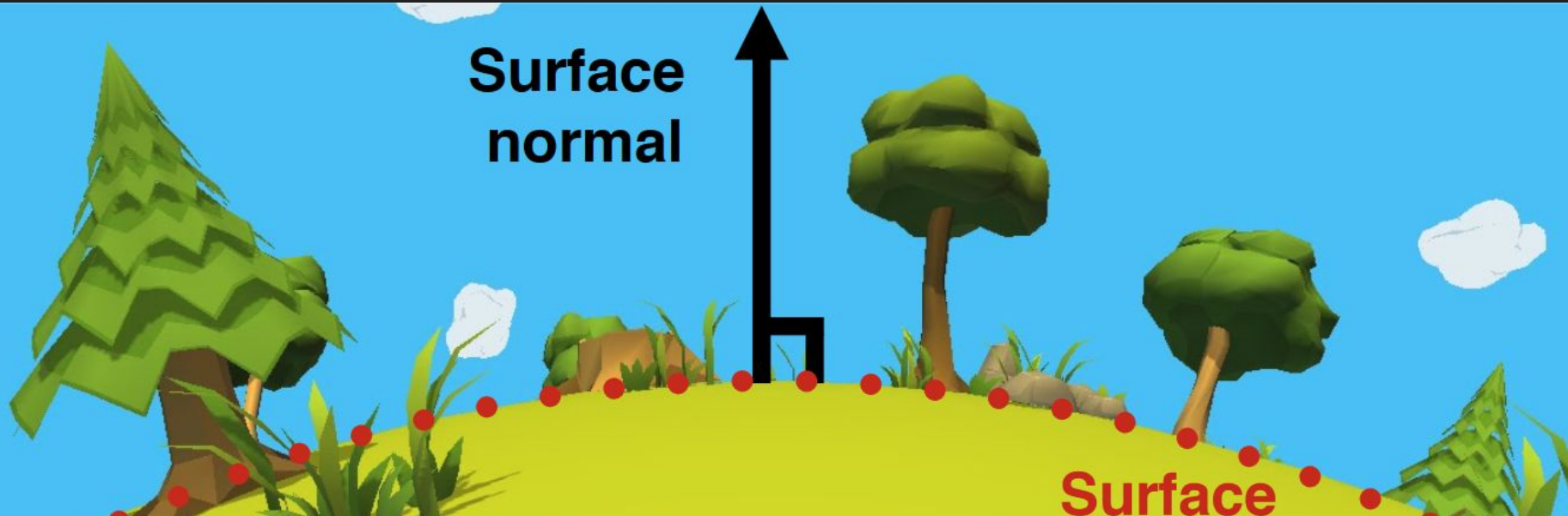
Cross product uses

- Finding a vector that is orthogonal to two other vectors
- Area of a parallelogram
- Finding the normal vector to a plane



Surface normal

- Aka normal vector
- Vector perpendicular to object in 2D or 3D
- Computed for each face (triangle) using any two edges a and b
- Usually unit vector $\mathbf{n} = \text{normalize}(a \times b)$



Reflection

Reflects a vector with regard to a surface normal

```
#include "glm/glm.hpp"
```

```
glm::vec3 d (6,-5,0);
```

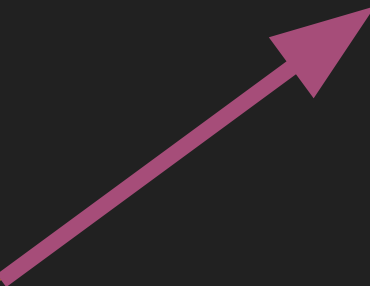
```
glm::vec3 n (0,1,0);
```

```
glm::vec3 r = glm::reflect(d,n); // (6,5,0)
```

**Incident
vector (d)**



**Reflection
vector (r)**



Axis-aligned rotations

Rotates around one coordinate axis.

```
#define GLM_ENABLE_EXPERIMENTAL
#include "glm/gtx/euler_angles.hpp"

float angle = 45 * DEG_2_RAD;
glm::mat4 m = glm::eulerAngleX(angle);
```

around x

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c & -s & 0 \\ 0 & s & c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

around y

$$\begin{bmatrix} c & 0 & s & 0 \\ 0 & 1 & 0 & 0 \\ -s & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

around z

$$\begin{bmatrix} c & -s & 0 & 0 \\ s & c & 0 & 0 \\ 0 & 0 & z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Directions vs Rotations

- A direction can tell us where an object is looking, but not how it's **oriented**
- Based on two non parallel directions we can compute a rotation



Orientation in 3D

Orientation in 3D has 3 degrees of freedom

Orientation can be expressed in multiple ways



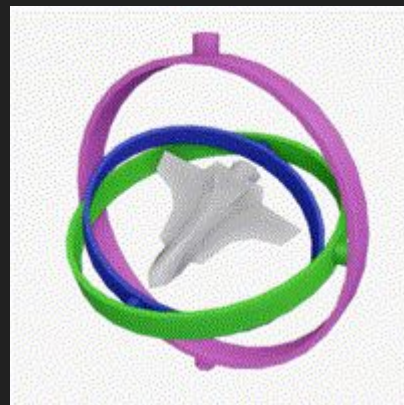
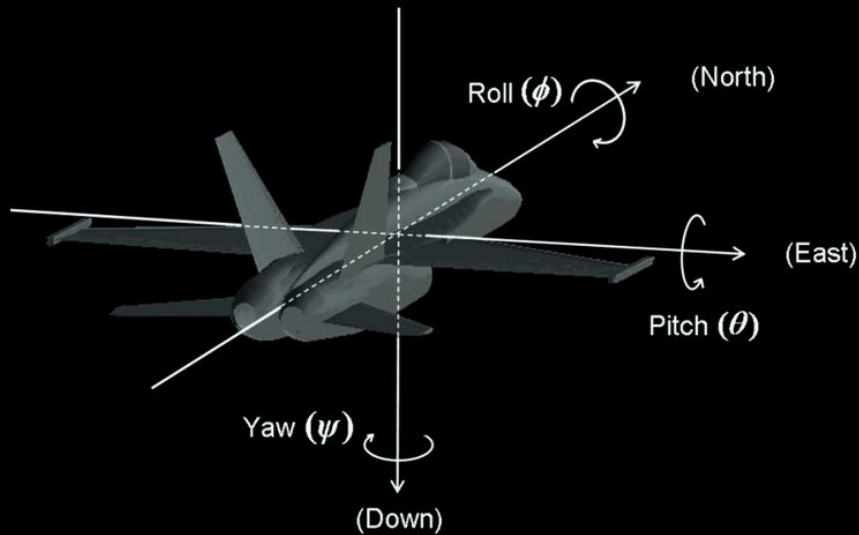
Euler's angles

Expresses rotation as a combination of rotations around x,y,z axis

The order which the rotations is carried out matters - but depends on usage

Gimbal lock problem (some combinations of rotations causes problems)

```
glm::mat4 r0 = glm::eulerAngleX(PI_HALF);  
  
glm::vec3 rot = [...];  
  
glm::mat3 r1 = glm::eulerAngleYXZ(  
    rot.y,  
    rot.x,  
    rot.z  
);
```

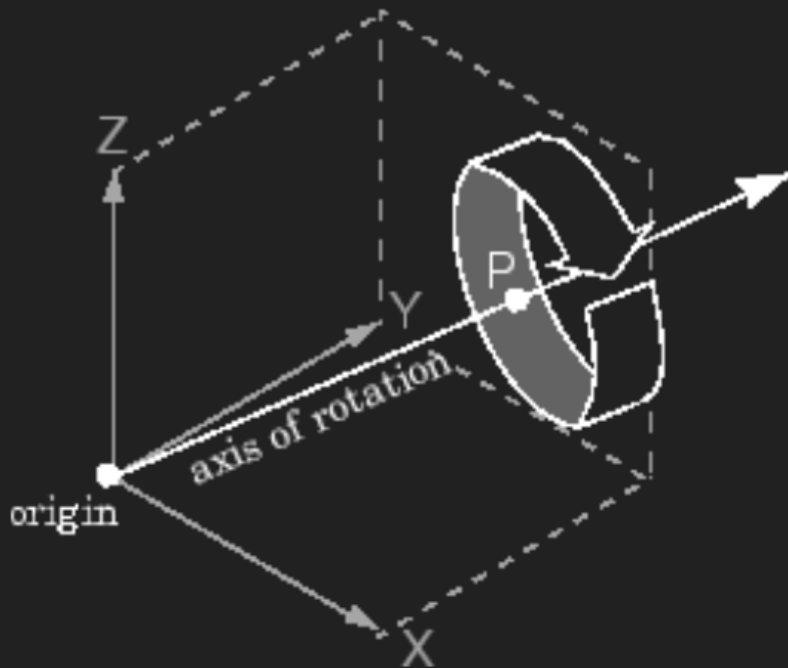


Rotate around an axis

- Another way to rotate around an arbitrary axis
- Sometimes useful for gameplay
- In GLM is implemented in the experimental namespace! (gtx)

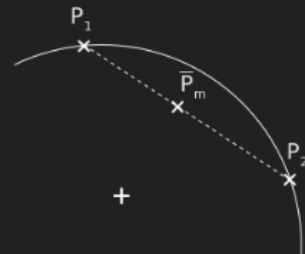
```
#define GLM_ENABLE_EXPERIMENTAL
#include "glm/gtx/transform.hpp"

glm::vec3 axis(1,1,1);
float angle = glm::radians(30.0f);
glm::mat4 rot = glm::rotate(angle, axis);
```

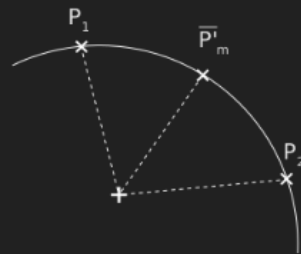


Quaternions

- Quaternions are mathematical constructs that can describe complex rotations without incurring in gimbal locks
- Composed of 4 numbers
 $q = [q_n \ q_o \ q_p \ q_d]$
- Easily convertible to vectors or matrices
- Allows correct rotation interpolations (called spherical linear interpolation or SLERP)
- Easy to invert and to rotate points



Linear interpolation



Spherical interpolation

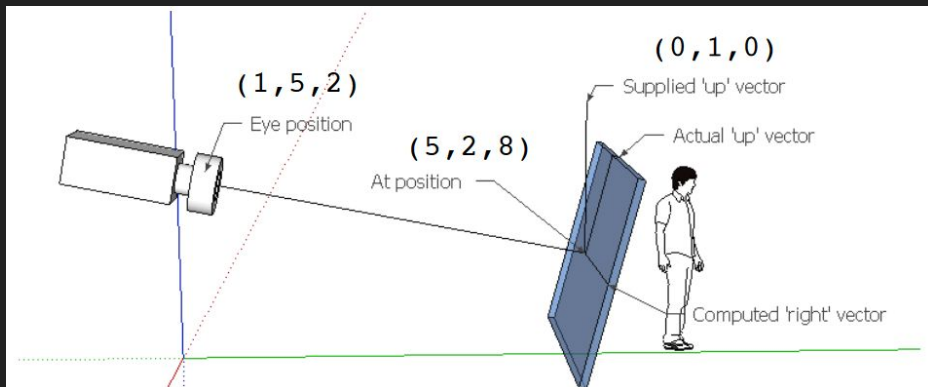
LookAt

Specifies observer's (usually camera, but can be used for anything really!):

- Position (3D point)
- Target (3D point)
- Up vector (3D direction)

```
#include "glm/gtc/matrix_transform.hpp"

glm::vec3 eye(1,5,2);
glm::vec3 at(5,2,8);
glm::vec3 up(0,1,0);
glm::mat4 view = glm::lookAt(eye,at,up);
```



Agenda

1. Additional math
2. 3D Rendering
3. 3D Animation

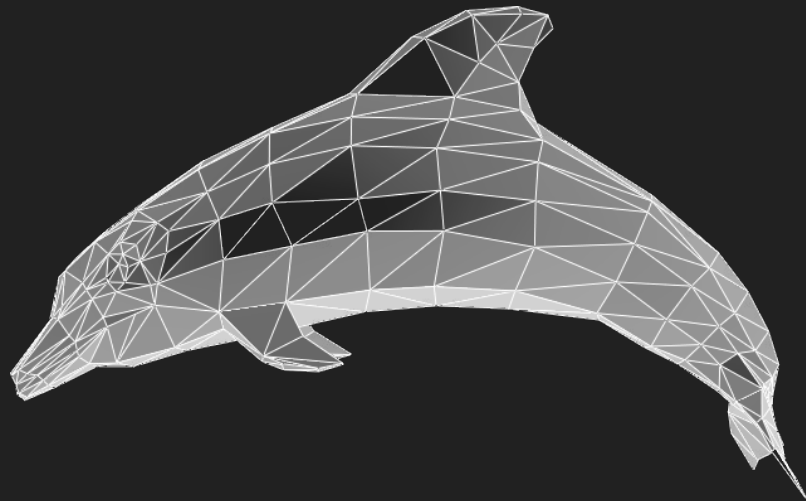
What is a Mesh?

A Mesh is a collection of data used to store and render 3D geometries.

Data needed to represent the structure of a 3D object:

- Vertices
- Edges
- Faces

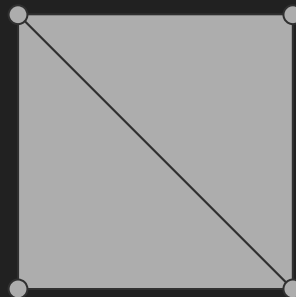
In 3D graphics, the only kind of faces that we are interested in are quads (because they are convenient to work with for 3D modeling and animation) and triangles (because that's convenient and performant for the GPU hardware)



Describing a triangular mesh

- List all the vertices in a table
- Simple and straightforward
- Vertex data often need to be duplicated, **wasting memory** (each floating point values takes 4 bytes)

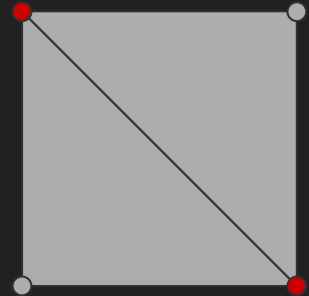
Vertex Position
(0, 0, 0)
(1, 0, 0)
(0, 1, 0)
(1, 0, 0)
(1, 1, 0)
(0, 1, 0)



Describing a triangular mesh

- List all the vertices in a table
- Simple and straightforward
- Vertex data often need to be duplicated, **wasting memory** (each floating point values takes 4 bytes)

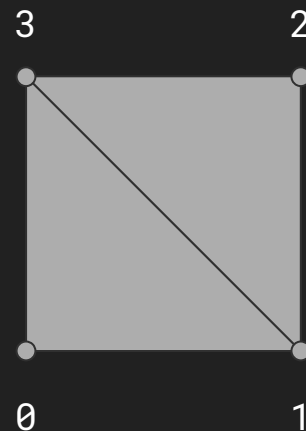
Vertex Position
(0, 0, 0)
(1, 0, 0)
(0, 1, 0)
(1, 0, 0)
(1, 1, 0)
(0, 1, 0)



Describing a triangular mesh

		Triangles
First triangle	[	0
		1
		2
Second triangle	[	1
		3
		2

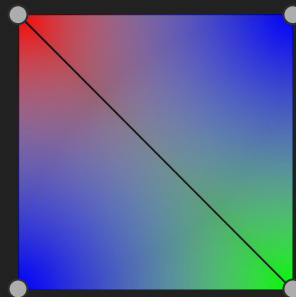
i	Vertex Position
0	(0, 0, 0)
1	(1, 0, 0)
2	(1, 1, 0)
3	(1, 0, 0)



Vertex data

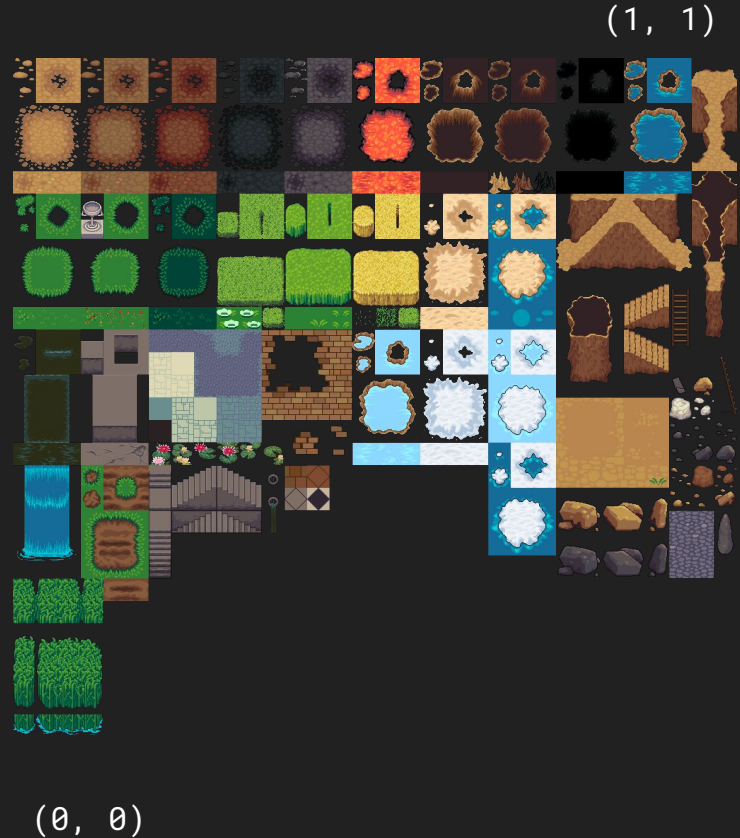
- A draw call can have multiple vertex attributes
- For other vertex attributes, values are interpolated across triangle

i	Position	Color	...
0	(0, 0, 0)	(0, 0, 1)	...
1	(1, 0, 0)	(0, 1, 0)	...
2	(0, 1, 0)	(0, 0, 1)	...
3	(1, 1, 0)	(1, 0, 0)	...



UV Coordinates

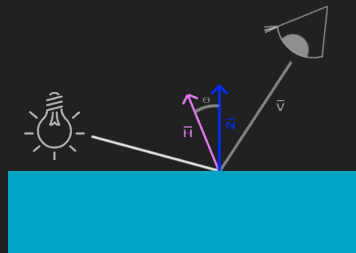
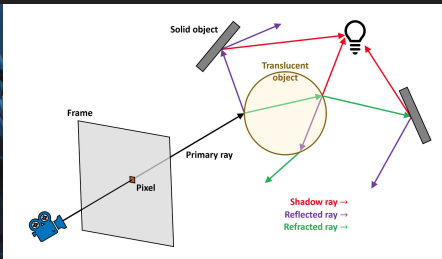
- One of the common vertex data
- Tells the engine which part of the desired texture to render on top of a given triangle
- Specified per vertex (and interpolated across a triangle)
- Difficult to map a 2D to a 3D surface (weird artifacts, similar to projecting Earth surface onto a 2D map)
- BUT! Easy to map a texture to a triangle (they are both 2D)



Lighting models

Global illumination

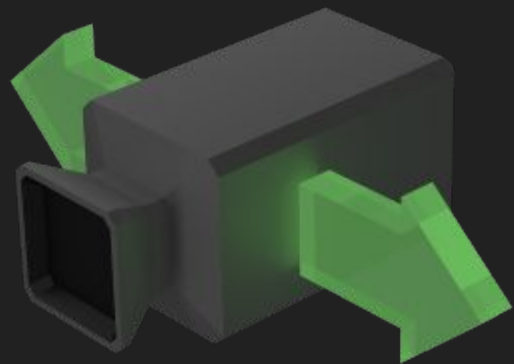
- Very realistic
- Simulate light rays bouncing between surfaces
- Used to be relegated to offline rendering (but nowadays it's a thing in real-time too!)
- Many different families of algorithms
 - Ray tracing
 - Path tracing
 - Photon mapping
 - And many more



Local illumination

- Used to be the standard for real-time rendering
- Traces light from light-source to object to camera, but ignores other objects
- Incredibly performant, but lacks certain physical characteristics
 - Shadows
 - Reflections
 - Subsurface scattering
 - Ambient occlusion
 - And much more
- But we became very good at faking them!

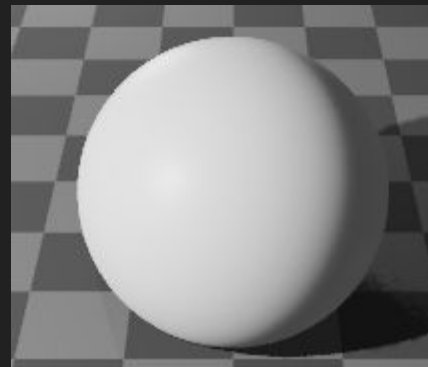
Inputs of a lighting model



Camera



Light



Object

Virtual cameras

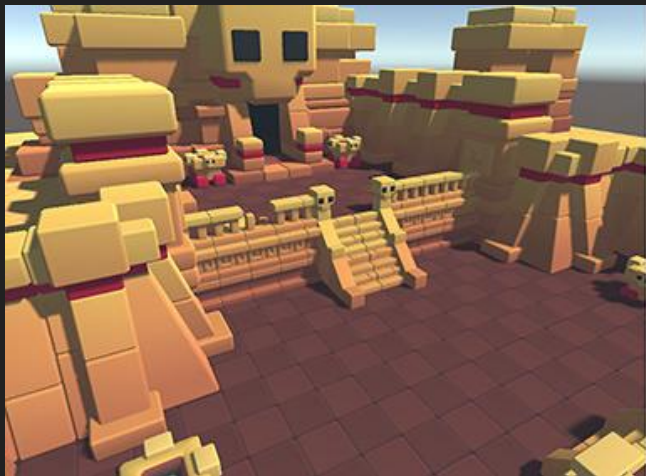
3D cameras, in addition of repositioning objects in a space where a renderer can display them, also needs to specify how objects are projected on a 2D screen. This is done with a projection matrix.

```
view = inverse(camera_global)
global = parent.global * local;
```

CPU

```
ndc    = proj * view * global * vertex
pixel =
/*
- perspective division
- scale by current window/screen
size
*/;
```

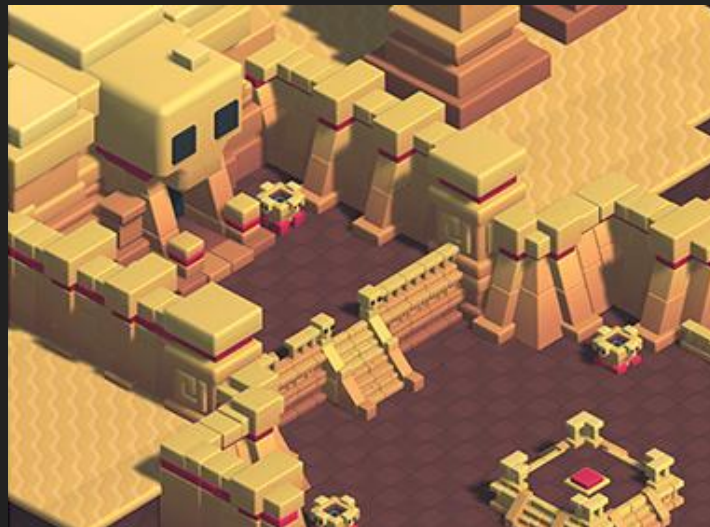
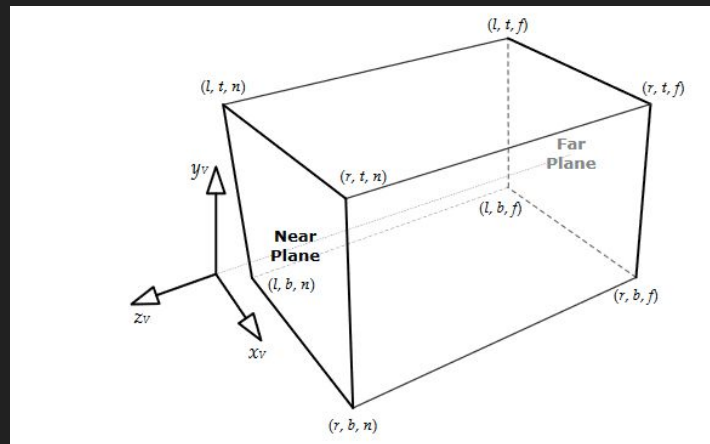
GPU



Orthographic camera

- Renders everything within a $[-1, 1]$ cube centered around the origin
- Objects appear the same size, disregarding of distance from camera
- Useful for 2D or isometric games

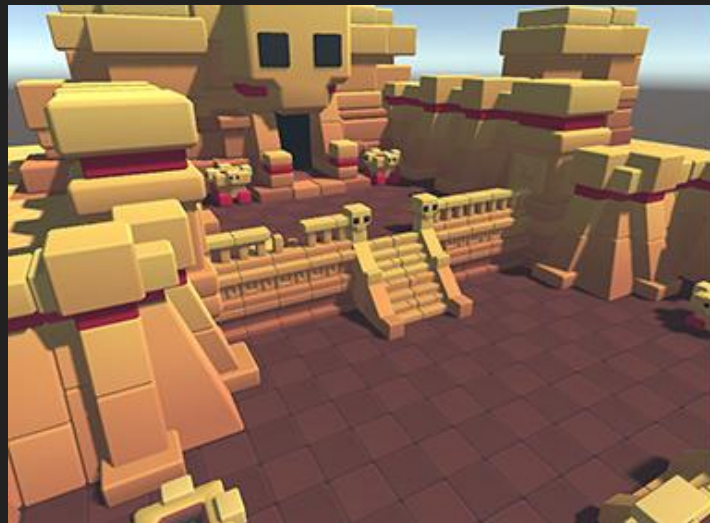
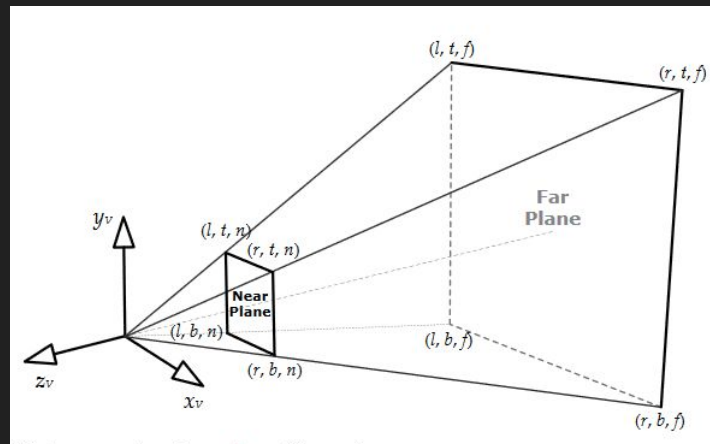
```
glm::mat4 proj;  
case CAMERA_TYPE_ORTHOGRAPHIC:  
{  
    float aspect = context->window_w / context->window_h;  
    float z_depth_half = (camera->far - camera->near) / 2.0f;  
    proj = glm::ortho(  
        -aspect * camera->zoom,  
        +aspect * camera->zoom,  
        -camera->zoom,  
        +camera->zoom,  
        -z_depth_half,  
        +z_depth_half  
    );  
    break;  
}
```



Perspective projection

- Creates a pyramid specified by:
 - Field of view angle (usually between 45 and 75 DEG)
 - Aspect ratio (of the window/screen)
 - Distance of the near plane (from camera pos)
 - Distance of the far plane (from camera pos)
- Mimics how closer objects seems bigger

```
glm::mat4 proj;  
case CAMERA_TYPE_PERSPECTIVE:  
{  
    float aspect = context->window_w / context->window_h;  
    proj = glm::perspective(  
        camera->fow, // field of view  
        aspect,      // aspect ratio  
        camera->near, // near plane  
        camera->far  // far plane  
    );  
    break;  
}
```



Lights

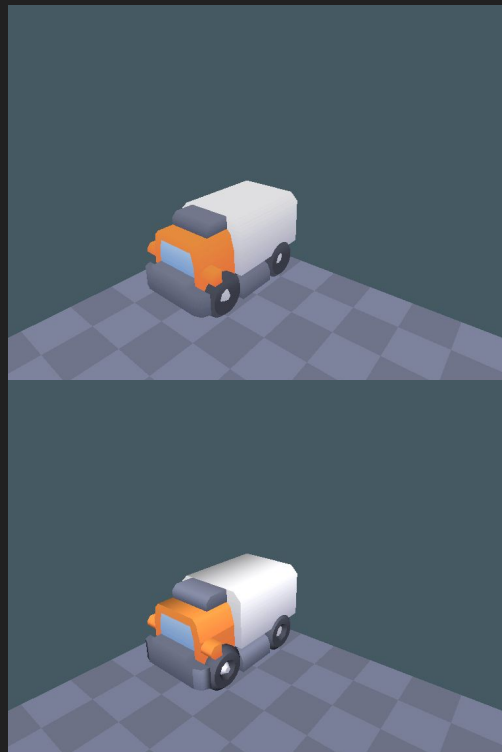
Used by shaders to properly alter an object's color to make it look 3D

Types

- Directional (sun)
- Point (torch, lightbulb)
- Spot (flashlight)
- Ambient (indirect light)

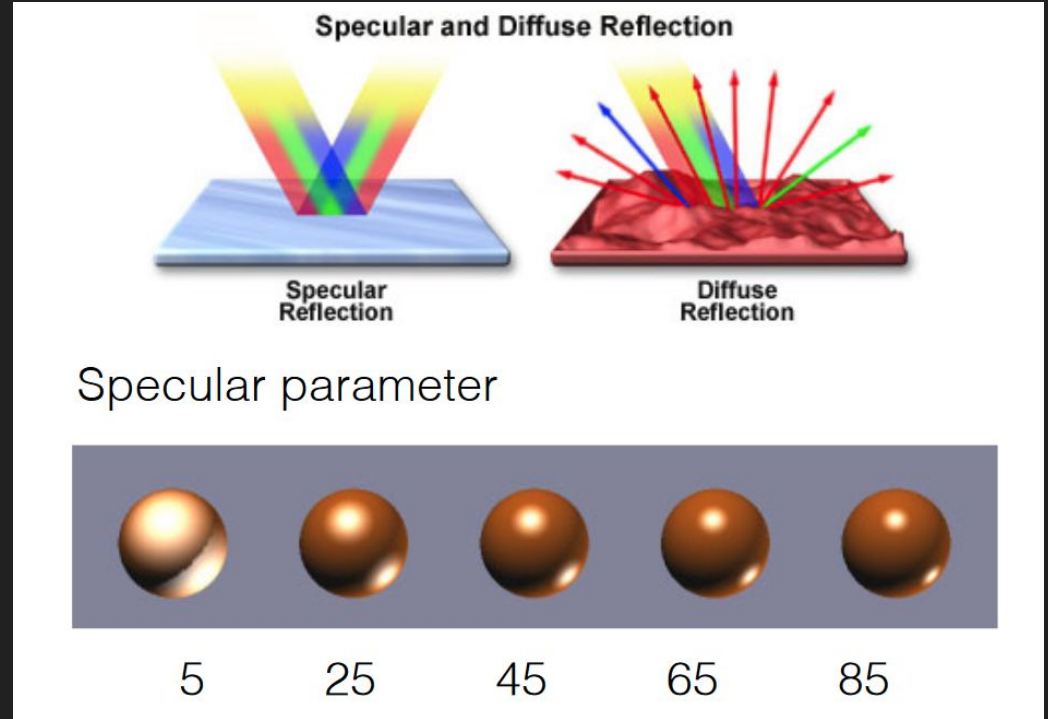
Parameters

- Color
- Intensity
- Direction (except point)
- Position (except directional)
- Attenuation (except directional)



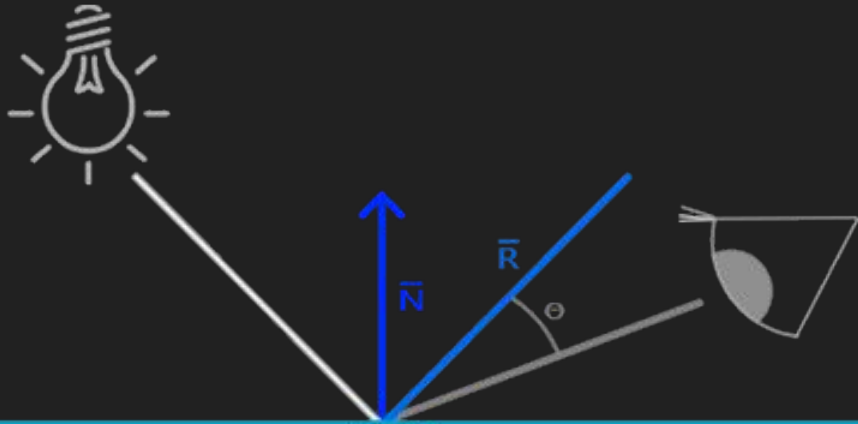
Objects and materials

- Different materials (metal, leather, plastic and so on) will react to light in different ways
- To model them we use an asset called a Material
- Material parameters
 - Shader
 - Shader's parameters
 - Auxiliary render parameters (depend on the engine)



Phong Model

Local model that can be computed fast
(but not physically accurate)

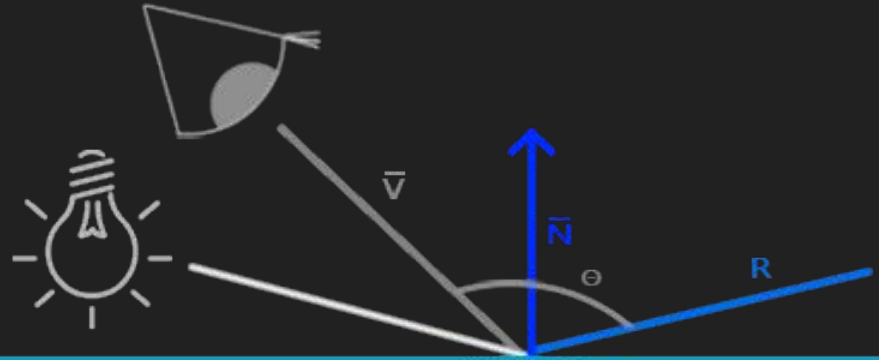


Three components

- Diffuse
- Specular
- Ambient

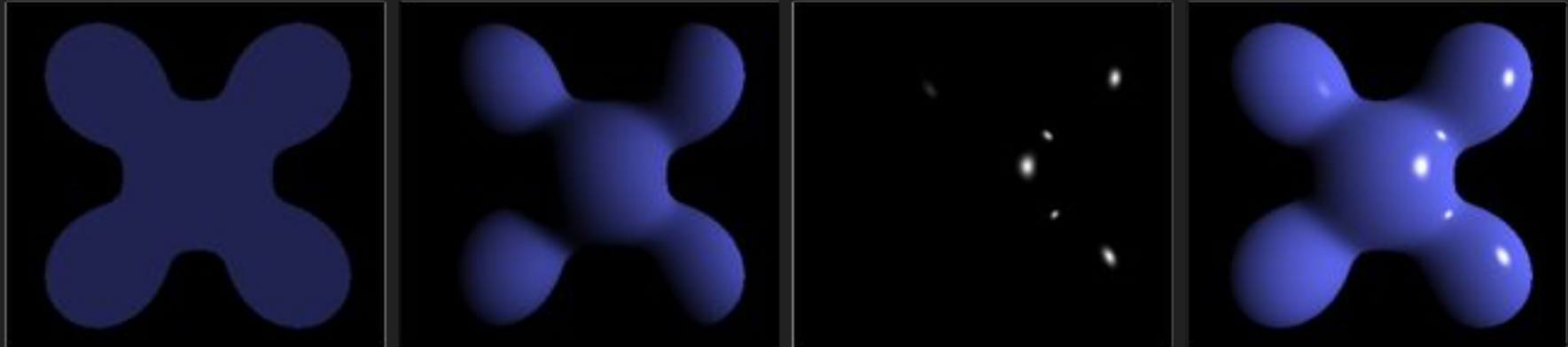
Uses four vectors

- To light source ($\vec{l}$)
- To viewer ($\vec{v}$)
- Normal ($\vec{n}$)
- Perfect reflector ($\vec{r}$)



Phong Model

- Surface color is determined by a product
 - Interpolated vertex color
 - A texture sample (based on interpolated UV coordinates)
- Final color = surface color * (ambient + diffuse light) + specular highlight
- Based on surface normal, incident light direction and eye position



Ambient

+

Diffuse

+

Specular

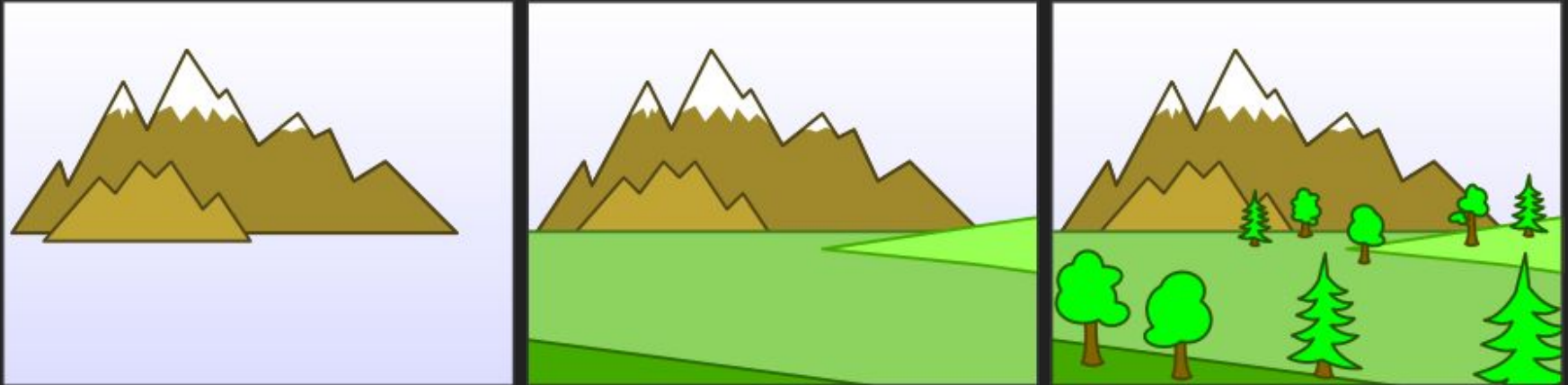
=

Phong Reflection

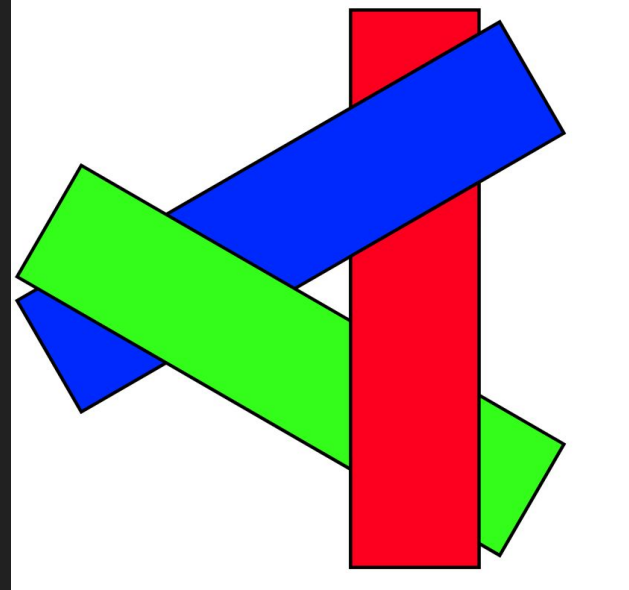
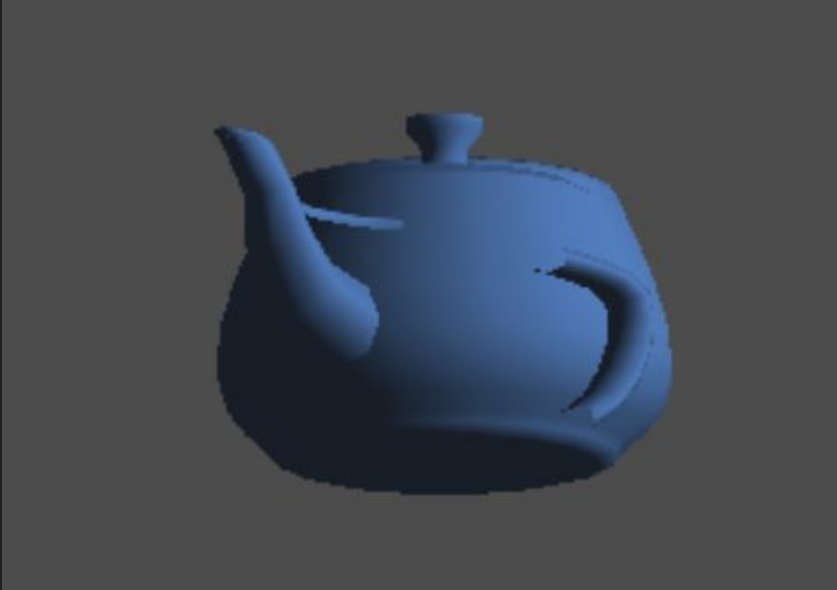
Rendering order

- Naive algorithm - draws geometry in the order it is rendered
- Useful for rendering sprites, where order is clearly defined

https://en.wikipedia.org/wiki/Painter's_algorithm



3D Painter's algorithm?

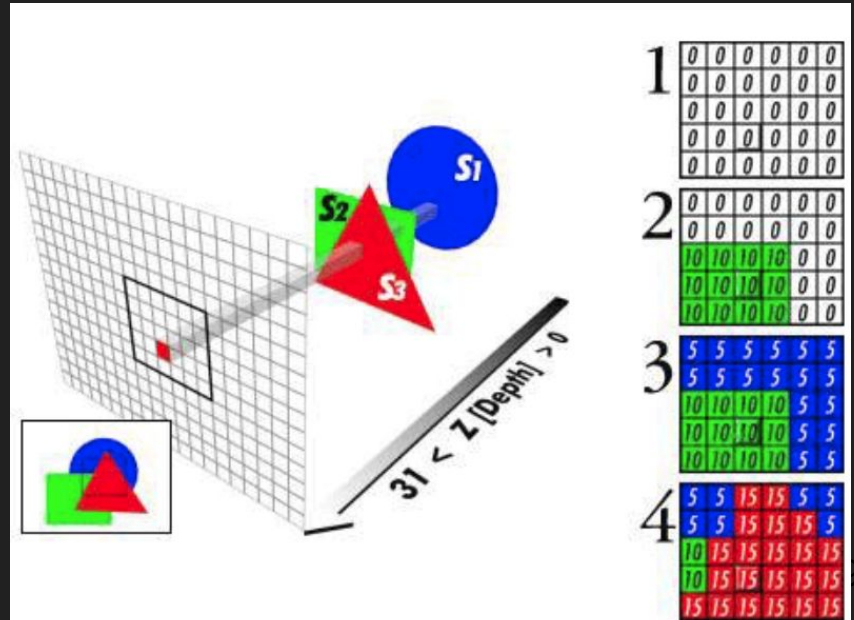


How to find the correct order for 3D objects?

Z-buffer

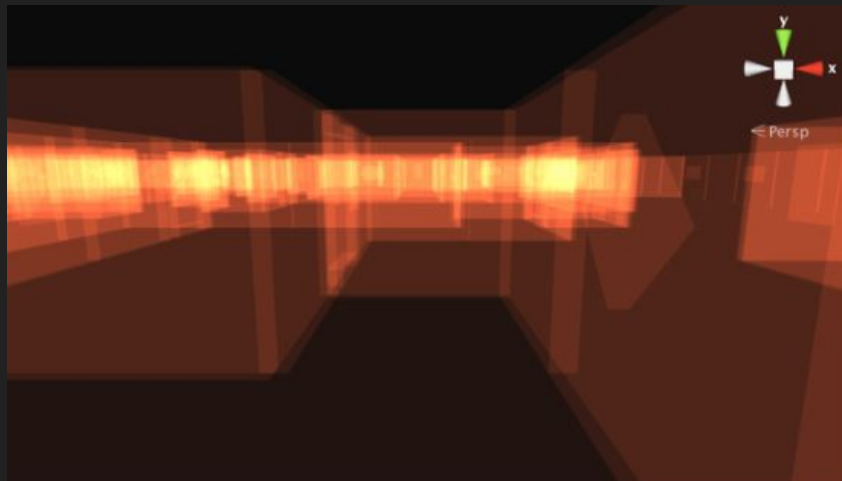
We can't really sort triangles based on their distance from the camera, but the intuition is still correct. We just need to move from triangles to... single pixels!

- For every pixel we store its distance from camera (interpolate the current triangle's vertex distance)
- This is stored in a 2D grid called z-buffer (or depth buffer)
- When rendering, we render only pixels closer than current value



Render order - opaque geometry

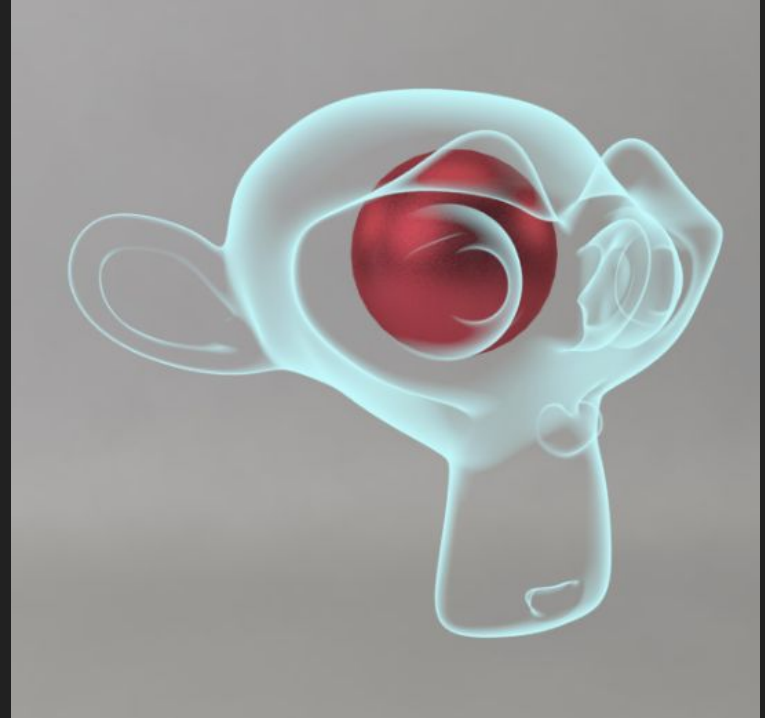
- We can avoid rendering the color of each pixel multiple times (overdraw) by ordering the draw calls by their distance to the camera
- Too slow to do triangle-by-triangle, but simply sorting by distance between camera and bounding box works well



Unity's overdraw visualization

Render order - transparent geometry

- Transparent objects must be rendered after all opaque geometry (since it blends with what is behind it)
- Must be sorted back to front
- Does not work well with Z-Buffers
- Usually much more expensive than opaque geometry!



Agenda

1. Additional math
2. 3D Rendering
3. 3D Animation

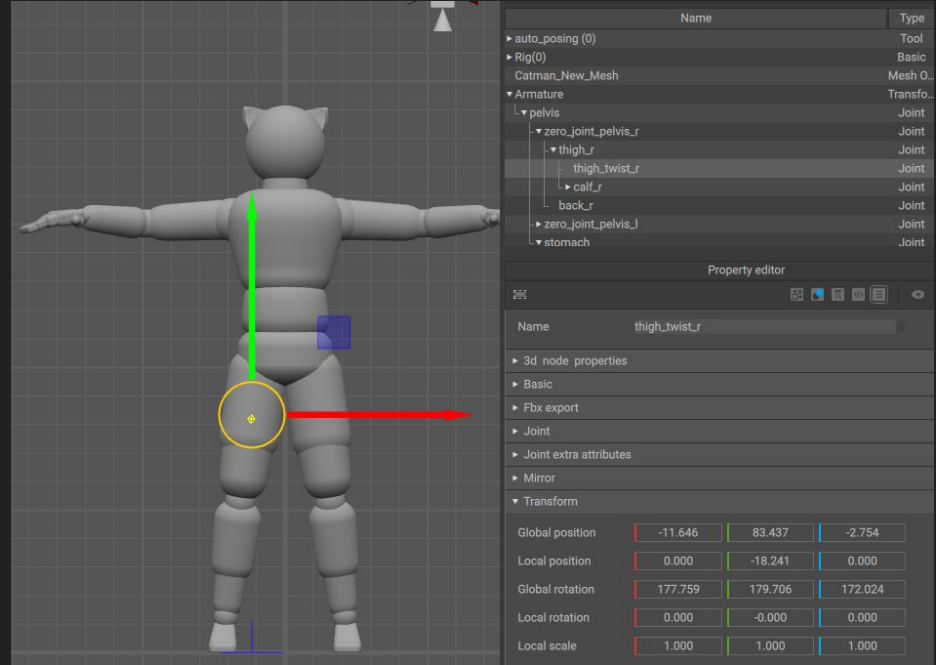
Rigid hierarchical animation

- Divide character into non-deformable parts in a scene hierarchy
- Animate the transforms of the hierarchy
- Works well for robots and similar rigid objects



Rigid hierarchical animation

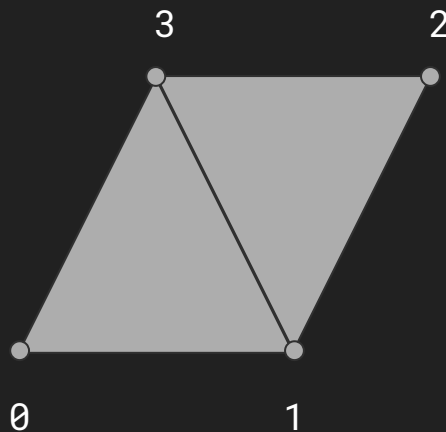
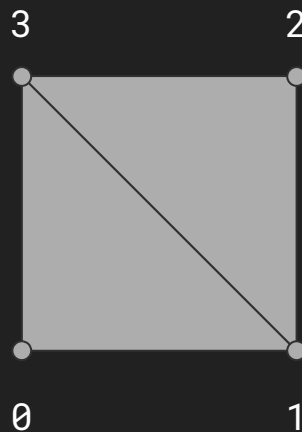
- Keyframes are defined with a transform matrix, usually called SQT (Scale, Quaternion rotation, Translate)
- In-betweens are computing interpolating two consecutive keyframes SQT transformations.



Per-vertex animation

- Simple way to do non-rigid animation
- Define keyframes by having multiple positions for each vertex
- In-between frames can be found using linear interpolation

i	pos keyframe 0	pos keyframe 1
0	(0, 0, 0)	(0, 0, 0)
1	(1, 0, 0)	(1, 0, 0)
2	(1, 1, 0)	(0.5, 1, 0)
3	(1, 0, 0)	(1.5, 1, 0)



Per-vertex animation problems

- Takes a lot of data
- Difficult to author
- Almost impossible to reuse



Per-vertex animation other usages

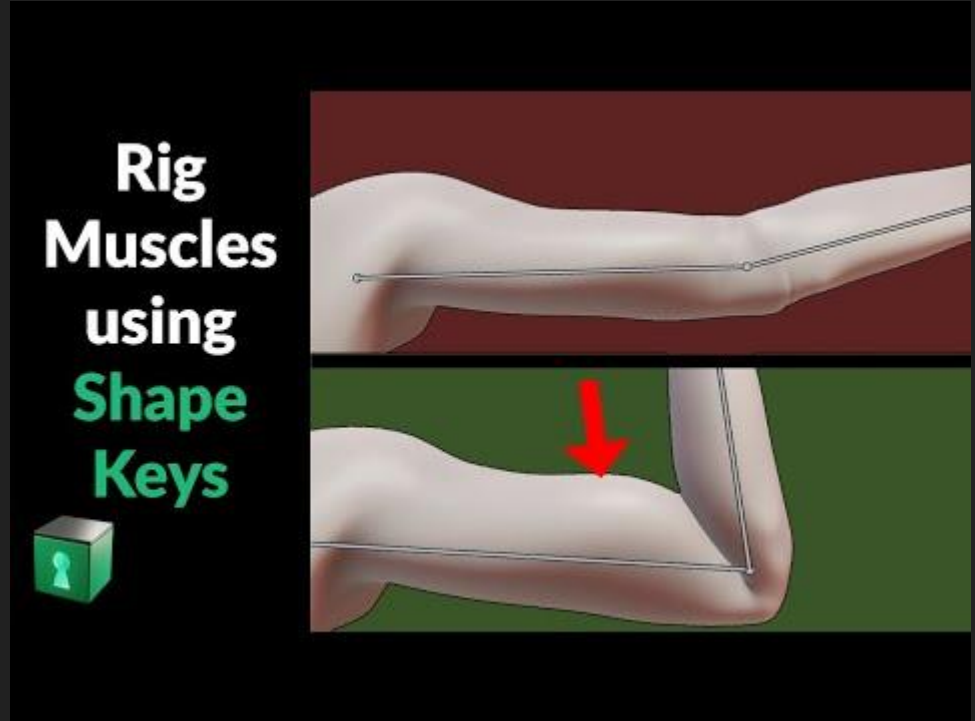
- Animated shaders
- Improvement of other kind of animation
- Cool game effects



<https://80.lv/articles/creating-next-gen-grass-in-ue4/>
<https://www.artstation.com/artwork/Vy0OAn>

Per-vertex animation other usages

- Animated shaders
- Improvement of other kind of animation
- Cool game effects



Per-vertex animation other usages

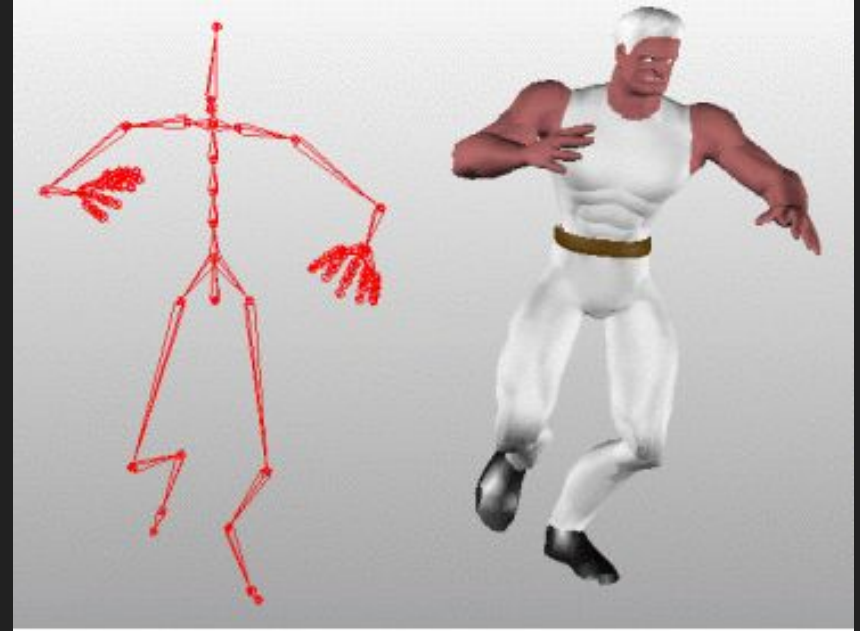
- Animated shaders
- Improvement of other kind of animation
- Cool game effects



<https://www.patreon.com/posts/22650450>

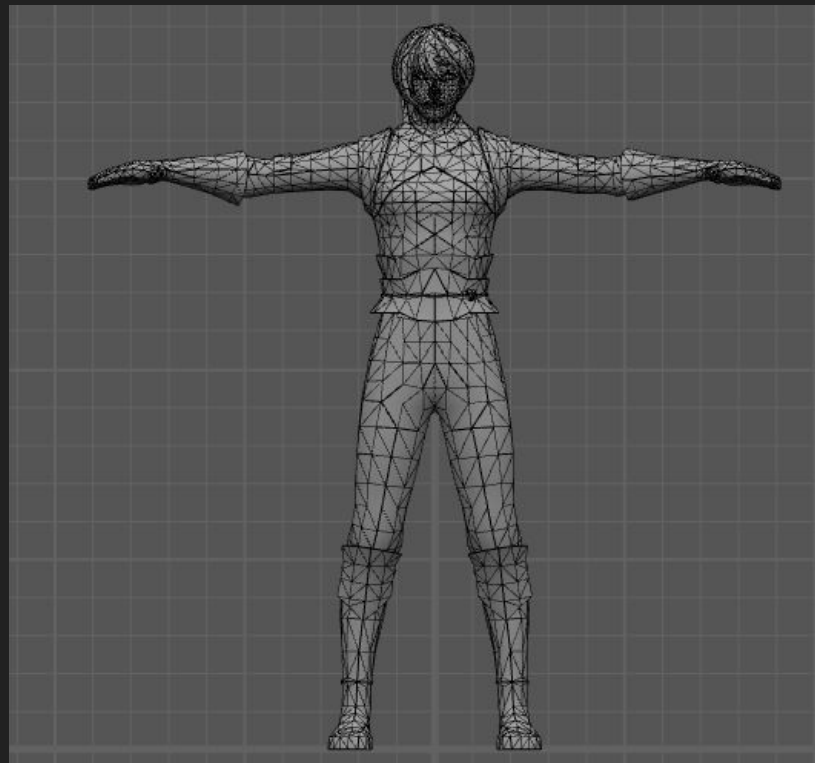
Skinned animation

- Can be seen as a combination of rigid and vertex animations
- We create a skeleton animated with rigid hierarchical animation.
- Each vertex is mapped to one or more 'bones' in the skeleton.
- We can find the animated position using the transformation of the skeleton bones.



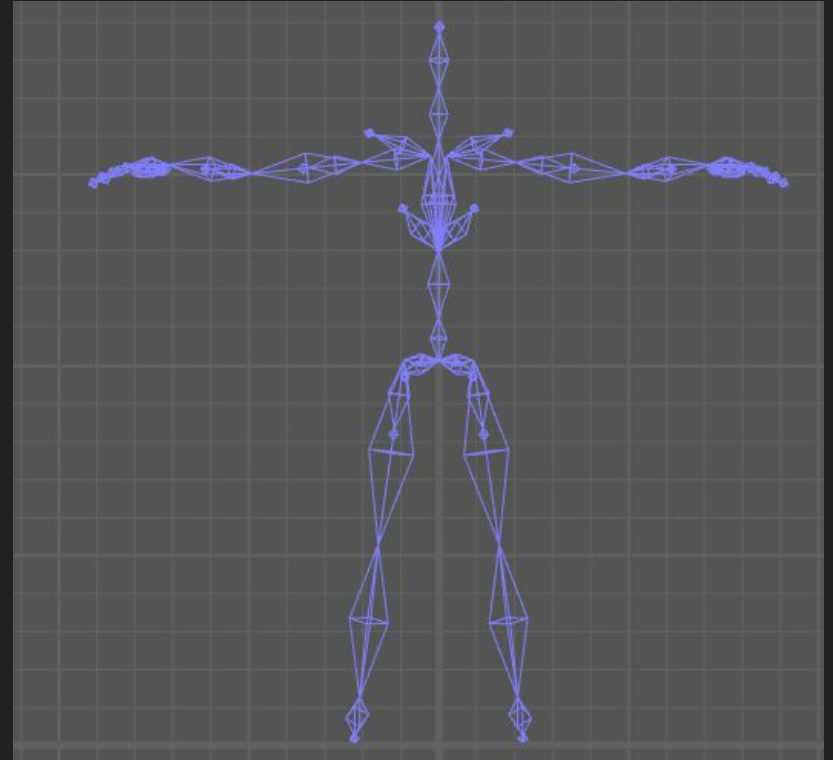
Bind pose

- Aka. reference pose / rest pose / Jesus pose / T-pose
- Arms to the side
- Feet apart
- Making mapping of vertices to bones easy



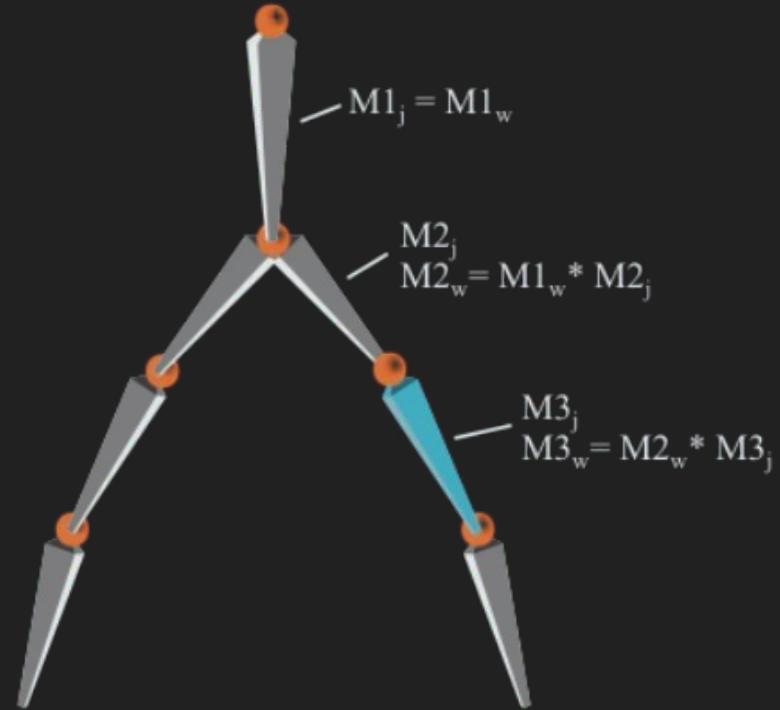
Skeleton hierarchy

- A simplified structure what is being modeled
- Humans are often modeled using same structure
- The hierarchy is put in the model



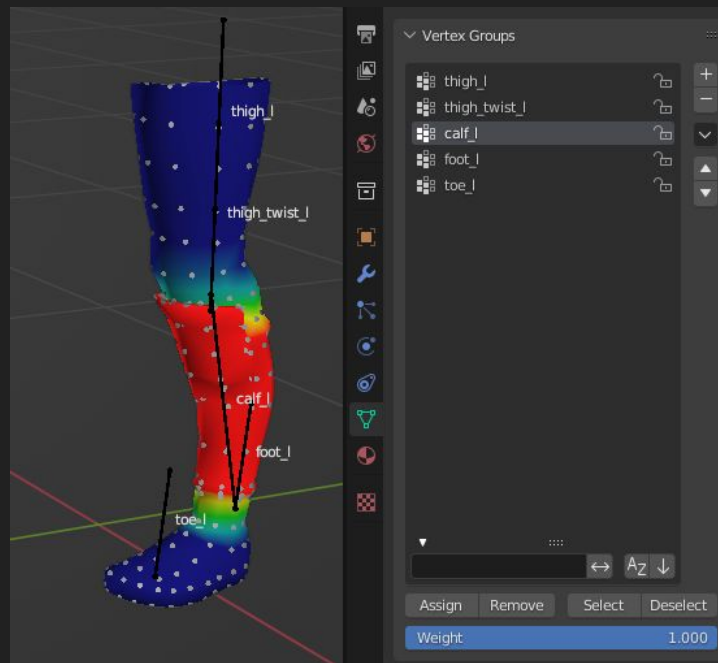
Skeleton hierarchy

- Joins and bones are the same thing (a bone is the space between two joins)
- Each join has
 - Join index
 - SQT transformation
 - Parent index



Rigging models

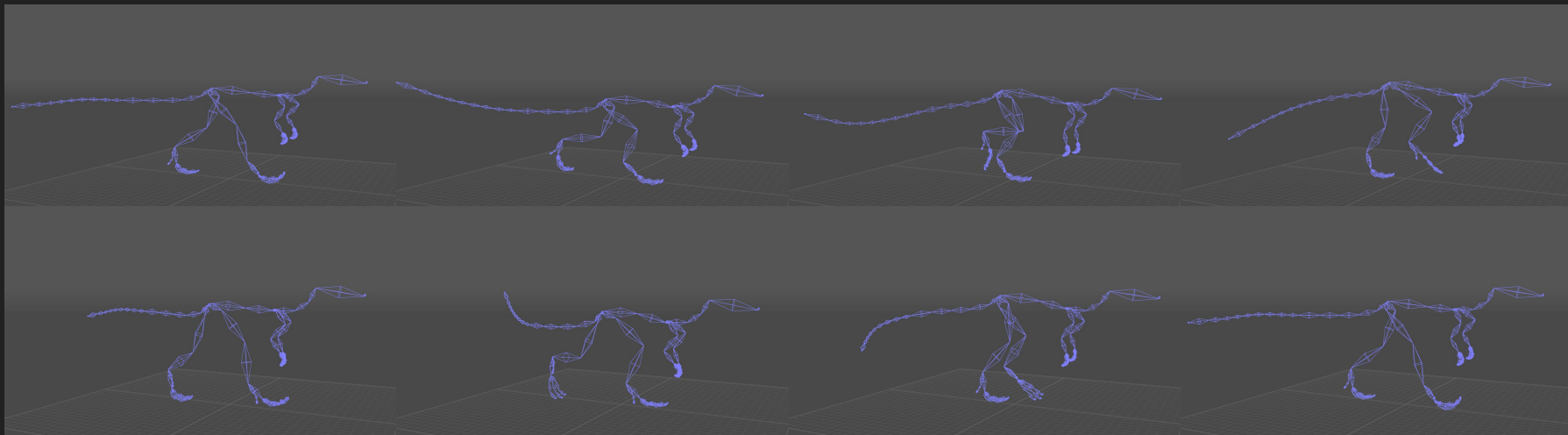
- Mapping each vertex to a number of bones
- For performance reasons, a fixed number of bones (usually, each vertex mapped to 4 bones)
- Each bone has a weight and the sum of all weights must be 1.0



```
// example of a vertex data structure
struct myVertexData {
    glm::vec3 position;
    glm::vec3 normal;
    glm::vec2 uv; // texture coordinates
    int bones[4];
    float boneWeight[4];
};
```

Keyframes

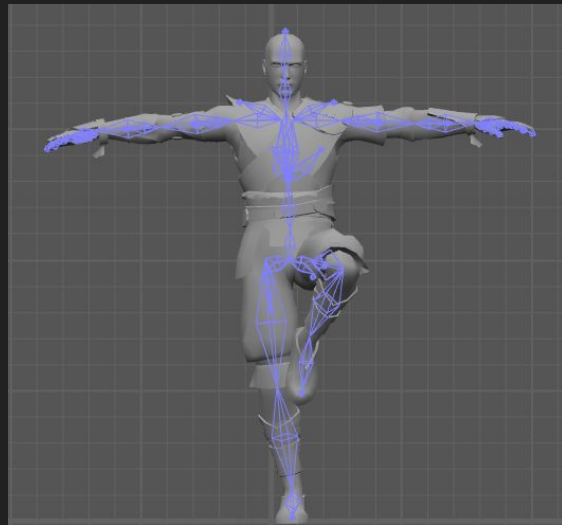
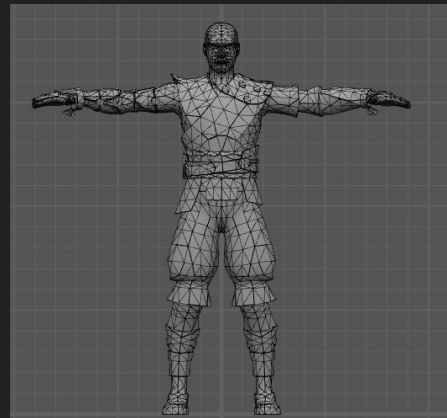
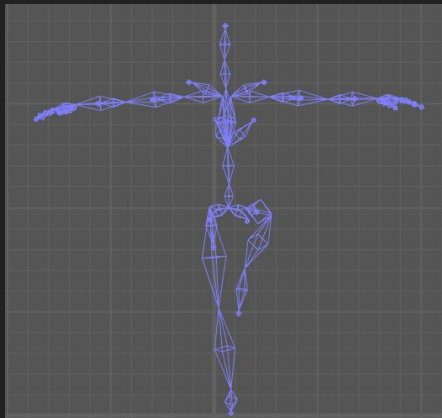
- Defined by modifying the transforms of the skeleton
- Storing the SQT of each join



Skinned animations

We can now compute the final destination of each vertex position

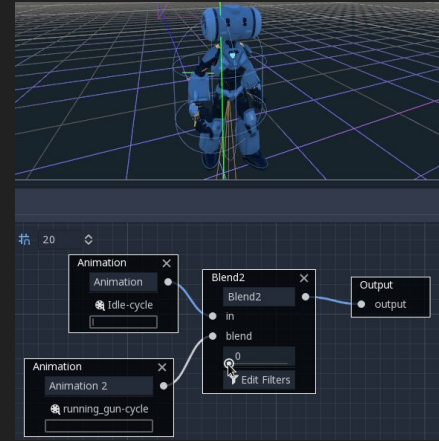
1. Compute skeleton position
2. Compute position of each vertex
 - a. For each associated joint, apply that joint transform to the vertex position
 - b. Average the 4 position obtained



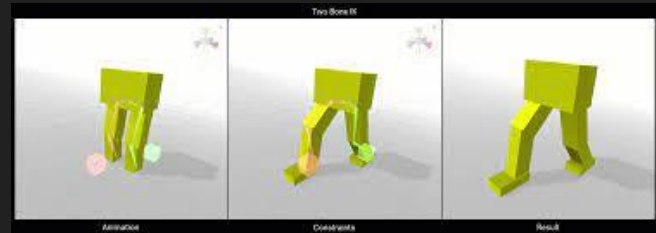
Advanced animation topics



Motion Capture
(digitalization of real performances)



Animation Trees
(smoothly blend between transitions)



Inverse Kinematics
(find position of bones based on targets)